Beyond the System Tick: Portable Time-Triggered Scheduling for Embedded RTOSes

Jonatan Sastre Hernandez*, Silviu S. Craciunas*†
*NXP Semiconductors, Vienna, Austria
†Technical University of Denmark, Kongens Lyngby, Denmark
Emails: jonatan.sastre@nxp.com, silviu.craciunas@nxp.com

***Abstract*—Embedded real-time operating systems (RTOSes) such as FreeRTOS/SafeRTOS, Zephyr, and PX5 are widely used in platforms ranging from automotive systems to industrial automation. They typically rely on fixed-priority (FP) scheduling and a periodic system tick for scheduling decisions. However, this tick-driven model lacks the deterministic guarantees of time-triggered (TT) scheduling and can introduce unnecessary overhead and coarse timing granularity. In this paper, we present a portable TT scheduling layer that retrofits deterministic, hardware-timer–driven execution into unmodified tick-based RTOSes without kernel changes. The approach preserves the native RTOS API while enabling sub-millisecond task activation, strict temporal isolation, and design-time control over scheduling behavior. By combining static TT scheduling with standard RTOS priority mechanisms, it adds TT benefits to existing RTOSes, effectively removes the tick as the time base for critical tasks, and maintains compatibility with legacy workloads. As a reference, we implemented our TTS approach on FreeRTOS and PX5. Our results show that tickless execution is not universally beneficial. Depending on the schedule and task set, a break-even point exists between tick-based and tickless modes. To support design-time decisions, we introduce an analytical overhead model characterizing when each mode is preferable. Hardware experiments comparing TT and FP scheduling further show that TT makes task start times a design-time property of the schedule table, keeping dispatch latency within a few microseconds of the scheduled slot even under transient overload, while FP scheduling exhibits priority-dependent, load-sensitive jitter tails.**

I. Introduction

Embedded real-time tick-based RTOSes such as FreeRTOS [1] (and its derivative SafeRTOS), Zephyr [2], and PX5 [3] are increasingly adopted in automotive, industrial, and edge computing systems due to their simplicity, availability, and portability [4]. In these domains, applications often require stringent real-time guarantees, temporal isolation, and predictable execution behavior. However, most commodity RTOSes rely on fixed-priority (FP) scheduling and a periodic system tick as the primary mechanism for driving scheduling decisions and timing services. In most tick-based RTOSes, scheduling decisions and timing services happen at tick boundaries and therefore inherit the temporal granularity of the system tick. Hence, the maximum uninterrupted execution time of a task and the precision of task-chain execution are fundamentally limited by the tick period. Moreover, the scheduler is invoked even when no rescheduling is required, which leads to increased execution overhead and reduced schedulability and responsiveness. Although fully tickless RTOSes eliminate these downsides, converting existing tick-based RTOSes to a fully tickless mode is complex and usually requires substantial kernel redesign.

Time-triggered (TT) scheduling [5] provides strong determinism and temporal isolation by enforcing task activations according to a statically defined schedule [6]–[10]. This enables analyzable and predictable system behavior, supports complex timing requirements, facilitates synchronization across computation and communication domains, and promotes compositional system design, making TT particularly attractive for highly critical real-time systems [11]. Moreover, TT scheduling offers a unique opportunity in tick-based RTOSes. In TT scheduling task activations are driven by a static schedule computed offline, making the release times of time-critical tasks known at design time. This enables the direct use of hardware timers to trigger task execution at precisely defined instants, with timing accuracy bounded by hardware resolution and interrupt latency rather than by the periodic system tick. Time-triggered scheduling has been widely adopted in domains with stringent timing and certification requirements, including automotive [9], industrial automation [12], and aerospace [5], and has also been explored in the context of tickless OSes, e.g., in the Linux kernel [13]. However, integrating TT execution semantics into widely deployed commodity RTOSes typically requires kernel modifications or rely on overhead-heavy user-space scheduling frameworks [13], limiting portability and industrial adoption.

In this paper, we (i) present a portable TT scheduling layer that retrofits time-triggered execution semantics into unmodified tick-based RTOSes without requiring kernel modification; (ii) preserve the native RTOS API and fixed-priority scheduling model while enabling hardware-timer–driven activation of time-critical tasks; (iii) develop a design-time analytical model that characterizes the trade-off between conventional tick-driven execution and TT-based scheduling; and (iv) experimentally evaluate the approach on real hardware platforms using FreeRTOS. For time-critical tasks, the proposed approach effectively eliminates the system tick as a time base when this yields performance or predictability benefits, while maintaining compatibility with legacy workloads.

After reviewing related work (Sec. II), we describe our TT scheduling layer in Sec. III. We introduce our overhead-driven design exploration in Sec. IV and show the experimental evaluation in Sec. V before concluding in Sec. VI.

II. Related work

TT scheduling has been introduced either as a middleware scheduling layer [14], [15] or explored in the context of general-purpose operating systems, for example through TT Linux kernel schedulers [13]. The LInux Testbed for MUlti-processor Scheduling in Real-Time Systems ($LITMUS^{RT}$) project [16] is an experimental extension to the Linux kernel implementing different partitioned and global priority-driven scheduling algorithms as well as table-driven scheduling (LRT_TABLE_DRIVEN). The work in [12] benchmarks a custom-built TT runtime system with synchronization for an industrial application that is not open-source.

Some tick-based RTOSes like FreeRTOS implement so-called *tickless idle* modes where the system tick is disabled during extended idle intervals [17]. While effective for reducing overhead in idle windows, these mechanisms do not fundamentally change the tick-based scheduling semantics. Other approaches (e.g. [18], embOS-Ultra [19]) change the kernel to remove the periodic system ticks with one-shot hardware-based timers that trigger only when specific events occur. In contrast, we aim to retrofit existing tick-based RTOSes without modifying the kernel to support true tickless execution. Moreover, we also provide an analytical model that characterizes the design-time break-even point between tick-based and tickless execution in unmodified RTOSes.

The runtime overhead of timer management and scheduling events has also been studied from an implementation perspective. Fröhlich et al. [20] study the trade-off between periodic and single-shot timer mechanisms in real-time embedded systems, showing that the relative efficiency of the two approaches depends on the hardware timer architecture. LazyTick [21] reduces job-release management overhead in RTOSes by avoiding unnecessary release processing, highlighting that the system tick remains a significant source of runtime cost. Brandenburg et al. [22] demonstrate that implementation choices such as periodic tick-driven scheduling versus event-driven scheduling can substantially affect overhead in real-time systems. In contrast to these works, we do not redesign the kernel timer subsystem or scheduler implementation. Instead, we retrofit hardware-timer-driven time-triggered activation as a portable layer on top of unmodified tick-based RTOSes, while preserving the native FP scheduling mechanisms for legacy workloads.

III. Time-Triggered Scheduling Layer

Time-triggered (TT) scheduling offers several well-established advantages for highly critical systems [6]–[10], [23], [24]. In TT scheduling, the static schedule table defines a sequence of slots up to the hyperperiod, each characterized by a start time, a duration, and the task assigned to execute in that slot. As the schedule is known at design time, all relevant activation events are known a priori. By enforcing task activations according to a statically defined schedule, it provides strict temporal isolation and supports correct-by-design approaches with strong determinism and analyzability. TT scheduling can accommodate complex timing constraints,

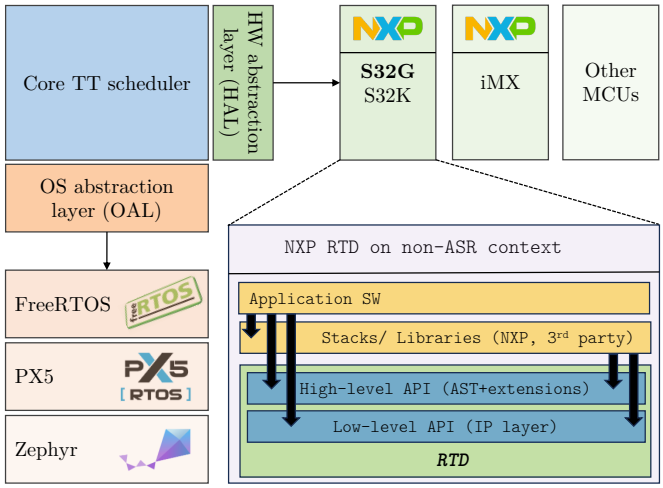


Fig. 1. Architecture of the TT scheduling layer

including cause-effect chains and jitter requirements, while often enabling higher system utilization compared to traditional fixed-priority approaches [13]. Furthermore, it improves predictability of runtime behavior such as preemptions and locking, simplifies synchronization with time-triggered communication, and supports compositional and incremental system integration across software components from different suppliers. In the context of tick-based RTOSes, it also offers an interesting opportunity to remove the reliance of the system tick for tick-based RTOSes like FreeRTOS and PX5. One way to remove the reliance on a system tick is by designing and implementing a tickless kernel. However, we aim to transform tick-based RTOS kernels into effectively tickless systems (i) in a non-intrusive way, (ii) preserving the native RTOS API without restricting original functionality, (iii) extending RTOS capabilities with TT scheduling while preserving fixed-priority legacy scheduling, and (iv) such that it is applicable to multiple existing RTOSes without modifying the kernel implementation, which is particularly important for safety certification and industrial adoption.

Our TT scheduling layer (cf. Figure 1) introduces a dedicated scheduler task that executes at the highest RTOS priority available only to the TT scheduler. Task execution is driven by hardware timers programmed on demand to the next slot boundary defined in the static schedule table, rather than by a periodic system tick. When a timer expires, the TT scheduler task is activated and enforces the static schedule by manipulating task priorities using standard RTOS mechanisms. At the beginning of a slot, the TT scheduler task raises the priority of the task assigned to that slot to the highest executable level (but below the scheduler task, which then self-suspends), ensuring immediate preemption of background tasks. At the end of the slot, or when the task voluntarily suspends, its priority is lowered to a predefined background level. The scheduler then programs the hardware timer for the next slot boundary. This mechanism generates exactly one interrupt per slot boundary, avoiding unnecessary periodic interrupts. While tick interrupts still occur to support legacy

services, critical TT tasks are driven by on-demand HW timers.

Since TT tasks never depend on the system tick for activation, the tick is effectively removed as a time base for TT tasks. Tick interrupts may still occur to support non-critical tasks and legacy services, but they do not influence the timing of TT execution. Hence, this results in functional tick elimination for time-critical tasks, e.g., by setting the tick to a very large value. The approach also provides deterministic overrun handling. If a TT task exceeds its allocated slot, the timer interrupt enforces descheduling by lowering the task's priority, regardless of whether it yields voluntarily. Overflowing tasks can use available slack as background tasks to finish execution later. This ensures temporal isolation and preserves the integrity of the static schedule correctness.

One main advantage of our design and implementation (cf. Figure 1) is the almost complete lack of specialized API usage in the application code to leverage the newly introduced tickless features for managing timing aspects. For example, when porting a task from traditional tick-based scheduling to event-driven task dispatching, the typical approach involves calling a dedicated API to wait for an event and resume the task upon its arrival. In contrast, our solution inherently handles these operations through its TT implementation. From the user perspective, system behaviour is not defined at runtime but specified offline through modeling tools and configurations.

We designed the layer to cover a broad set of hardware architectures and RTOS variants, thereby maximizing portability and general applicability. Hence, we structure it around two abstraction layers, namely the OS abstraction layer (TTS OAL) and the hardware abstraction layer (TTS HAL) that guarantee portability and seamless integration without necessitating modifications to the original operating system components (cf. Fig 1). These layers establish comprehensive coordination between the OS and the underlying hardware, while introducing a non-intrusive user API that enables initialization of the task scheduling mechanisms through a single function call. Importantly, full compatibility with the native RTOS API is maintained, allowing unmodified use of existing functionalities. In addition, applications are transparently managed by the TTS scheduler without requiring an alternative or dedicated API, unlike existing solutions, which impose direct handling of timing constraints at the application level.

A. Hardware Abstraction Layer (HAL)

The HAL is responsible for abstracting access to the hardware resources on which the TTS depends, primarily an external hardware timer used to track time and to trigger time-based scheduling events, such as scheduling-related decision logic and context switches. The timer functionality is limited and generally consistent across different hardware timer implementations. To simplify the development and scalability of the Core TTS, this layer provides a common API that exposes those generic functions in a uniform manner. For this paper we implemented the abstraction layers for the NXP S32G family reference design board (see Section V). The S32G SoC includes the System Timers Module (STM), which provides multiple timer instances where each timer consists of a counter capable of operating in one-shot or continuous mode, and four compare channels for setting up to four timeout values. When the counter reaches the value configured in any of the compare channels, an interrupt request is raised.

Some timers allow timeout values to be specified either as relative or absolute time references. We prefer absolute values, as they are more convenient. Note that, when a hardware timer cannot be configured with absolute time values, such behavior can be emulated in software, ensuring that the abstraction layer remains valid regardless of the specific hardware timer implementation.

B. OS Abstraction Layer (OAL)

The OAL provides a common API for performing the OS-level operations required for the TT scheduler (TTS) to function. Our TTS enforces strict timing and execution order of the tasks by dynamically adjusting task priorities through OS APIs. Although the API may vary across different OS implementations, the required functionality is limited and well-defined, which enables a concise abstraction layer that can be efficiently adapted to different RTOS environments. Similarly to the HAL layer, it allows decoupling the Core TTS logic from OS-specific mechanisms, ensuring portability across different RTOS families and simplifying the integration of additional OS backends in the future.

E.g., some RTOS API functions used in FreeRTOS OAL:

- `xTaskNotifyGive()` / `ulTaskNotifyTake()`, used to suspend and resume tasks at synchronization checkpoints.
- `vTaskPrioritySet()`, used for dynamically adjusting task priorities.

We identified a comparable set of native RTOS API functions in other RTOS implementations, such as PX5, suggesting that the OAL concept remains valid. We note that some OS designs like OSEK do not support dynamic modification of task priorities. In such systems it still is possible to impose a TT design by avoiding some patterns (e.g., τ_A preempts τ_B and τ_B preempts τ_A) in the resulting schedule.

C. Core Time–Triggered Scheduler (TTS)

This is the main layer that implements the core part of our concept. All decision logic and task management are performed here, using the required hardware and software resources through the corresponding abstraction layers described above. Overall, the proposed TT scheduling layer enables deterministic, sub-millisecond execution on existing tick-based RTOSes, with timing precision bounded only by hardware timer resolution and interrupt latency. By avoiding kernel modifications and specialized APIs, the approach combines the determinism of time-triggered systems with the flexibility and ecosystem support of general-purpose RTOSes.

IV. Overhead-driven design exploration

Typical tick periods are set to 1 ms, and many systems (e.g., FreeRTOS) do not support higher frequencies even when the underlying hardware would allow it. Moreover, even when

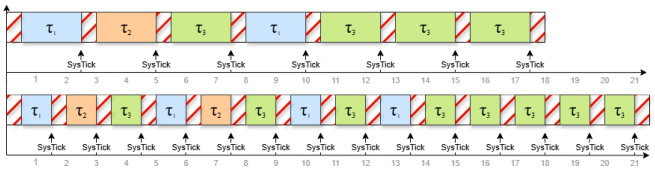


Fig. 2. FreeRTOS default, tick-based FP, RR, Timeslicing scheduling

Fig. 3. Fully cooperative scheduling

Fig. 4. Tick-less TTS based scheduling

higher frequencies are available they have the downside that they increase interrupt and kernel overhead, reducing the CPU time available for tasks. For example, in [12], a custom-built time-triggered OS running on a TMS570 MCU with an Arm Cortex-R4F at 180 MHz experiences between 400 ns and 4 μs of tick overhead, leading to up to 20% overhead at high time granularity (cf. Figure 6 in [12]). In our experiments, the tick overhead of FreeRTOS is approximately 3.5 μs for a task context switch initiated by the tick interrupt handler on a Cortex-M7 of an S32G274A running at 400 MHz (cf. Sec. V). This overhead degrades responsiveness and makes tick-based RTOSes less suitable for systems with high utilization or for tasks requiring sub-millisecond precision. More fundamentally, the periodic tick limits the ability to provide design-time control over task activation patterns, which is required in predictable real-time systems (see Section V-D).

Typically, the overhead for TT scheduling is bigger than the pure tick overhead but is invoked much less often (cf. Figures 2, 4). This exposes a fundamental trade-off between tick-based and tickless systems which can be transformed into a design optimization problem that informs architectural decisions. Hence, we aim to decide offline if it is more beneficial in terms of overhead to use a tick-based TT schedule or if the original tick-based implementation. Note that we focus on quantifying scheduling overhead as a first-order design metric, independent of blocking and interference effects. While classical schedulability analysis determines feasibility, overhead analysis informs architectural trade-offs between tick-driven and TT-driven execution modes.

We note that e.g., in FreeRTOS, the FP scheduler can be configured to run in FIFO or round-robin (RR) mode when selecting tasks from the same priority queue. RR is triggered by the system tick and imposes a large number of context switches (cf. Figure 2). FIFO essentially enables a cooperative mode in which the system tick can be removed most of the time (cf. Figure 3) but this mode relies on cooperative mode, losing the ability to enforce timing guarantees. This loss of preemptive control is particularly problematic in mixed-criticality systems, where safety-relevant real-time tasks must be protected from interference and must meet strict timing requirements. We therefore perform our design-time analysis for the round-robin mode.

To enable this design optimization, we aim to quantify the overhead in both systems. We consider a set of periodic tasks $\tau = \{\tau_1, \ldots, \tau_n\}$ where each task τ_i is characterized by the tuple $\langle \phi_i, C_i, T_i, D_i \rangle$, denoting its offset, WCET, period, and relative deadline, respectively. We assume a feasible periodic task set with constrained deadlines ($D_i \leq T_i$). For the purpose of the analysis, we assume that tasks have no locks and there are no external interrupt-driven releases. In the proposed TT-based approach, scheduling overhead occurs only at slot boundaries. Let m be the number of TT slots within one hyperperiod H for the synthetized static schedule and δ_{TT}^{min} and δ_{TT}^{max} the min and max overhead of one TT scheduling event, respectively. Note that any overhead incurred during reclaimed slack when critical TT tasks finish early is not charged to the critical workload, and therefore does not affect the schedulability or timing of critical tasks. The total TT overhead per hyperperiod H is thus bounded by:

$$m \cdot \delta_{TT}^{min} \leq \Delta_{TT}(H) \leq m \cdot \delta_{TT}^{max}.$$

Let T_{tick} denote the system tick period and δ_{tick}^{min} and δ_{tick}^{max} the minimum and maximum execution cost of one tick interrupt, respectively. For generality, we do not assume that H is an integer multiple of the system tick T_{tick}. In a tick-based FP system with round-robin execution, overhead does not come only from the system tick but also from scheduling decisions that are necessary at task releases and when tasks finish their execution or are preempted. Scheduling events are therefore caused by (i) periodic tick processing, (ii) task releases observed at tick boundaries, and (iii) job completions, which may trigger an immediate dispatch between ticks, as in common RTOSes such as FreeRTOS and Zephyr. The number of tick interrupts that occur during any interval of length H depends on the relative phase but differs by at most one tick:

$$N_{tick}(H) \in \left\{ \left\lfloor \frac{H}{T_{tick}} \right\rfloor, \left\lceil \frac{H}{T_{tick}} \right\rceil \right\}.$$

We model tick bookkeeping and FP scheduling decisions as separate costs. The tick bookkeeping cost is incurred at every system tick. The FP scheduling cost is incurred once for each distinct release instant, even if multiple jobs release simultaneously. Releases are counted as $N_{rel}(H) = \left| \bigcup_{i=1}^{n} \{\phi_i + kT_i \mid k \in \mathbb{N}_0,\ 0 \leq \phi_i + kT_i < H\} \right|$. Moreover, in the best case job completions align with releases such that no additional scheduling decision is triggered by a completion, and that all releases fall within the hyperperiod window (i.e., no carry-in/out effects). Hence, the tick-based overhead over one hyperperiod H is bounded from below by:

$$\Delta_{TB}(H) \geq \left\lfloor \frac{H}{T_{tick}} \right\rfloor \delta_{tick}^{\min} + N_{rel}(H)\delta_{FP}^{min}.$$

In the worst case for the round-robin case, in addition to the tick interrupt that executes at every tick, a scheduling decision may be taken at every tick, potentially resulting in preemption. For a generic worst-case task set, every tick may coincide with at least one release; therefore we upper-bound the release-driven FP scheduling cost by one FP decision per tick. Additionally, job completions may occur between ticks and may trigger an immediate scheduling decision and context switch. Furthermore, in constrained deadline feasible task sets, each task τ_i can contribute at most one carry-in completion at the beginning of an interval (including an interval of length H). Hence, the maximum number of job completions of the task set τ within an interval of length H is bounded by:

$$\sum_{i=1}^{n} \left(\left\lceil \frac{H}{T_i} \right\rceil + 1 \right).$$

Combining the above, the scheduling overhead of a tick-based FP system over any interval of length H is

$$\Delta_{TB}(H) \leq \left\lceil \frac{H}{T_{tick}} \right\rceil (\delta_{tick}^{max} + \delta_{FP}^{max}) + \sum_{i=1}^{n} \left(\left\lceil \frac{H}{T_i} \right\rceil + 1 \right) \delta_{FP}^{max}$$

We say that one approach is *certainly better* than the other if its worst-case scheduling overhead is no larger than the best-case scheduling overhead of the alternative.

Tickless TT execution certainly dominates tick-based round-robin FP scheduling (Figure 2) in terms of overhead if

$$m \cdot \delta_{TT}^{max} \leq \left\lfloor \frac{H}{T_{tick}} \right\rfloor \delta_{tick}^{\min} + N_{rel}(H) \delta_{FP}^{min}.$$

Conversely, tick-based RR FP scheduling is certainly better than tickless TT scheduling in terms of overhead if:

$$\left\lceil \frac{H}{T_{tick}} \right\rceil (\delta_{tick}^{max} + \delta_{FP}^{max}) + \sum_{i=1}^{n} \left(\left\lceil \frac{H}{T_i} \right\rceil + 1 \right) \delta_{FP}^{max} \leq m \cdot \delta_{TT}^{min}.$$

The analysis presented above focuses exclusively on scheduling overhead and does not incorporate blocking due to shared resources, synchronization primitives, interrupt interference, or priority inversion effects. Furthermore, it does not constitute a schedulability analysis. Instead, it provides a design-time comparison framework for estimating the relative overhead introduced by tick-based and TT-based execution under identical task sets. Standard schedulability analysis techniques (e.g., response-time analysis or table-driven feasibility analysis) must still be applied independently to verify timing correctness of the resulting system configuration.

V. Experiments

All experiments were performed using an NXP-S32G-VNP-RDB2 reference design board, which integrates three ARM Cortex-M7 microcontrollers. Each core operates at up to 400 MHz and features tightly coupled memory (TCM), enabling deterministic real-time execution. The workload is a set of synthetic periodic tasks whose payload is a time-bounded busy-wait loop polled against the SoC's hardware timer (STM counter): each task τ_i is configured with a per-job time budget C_i where the busy-wait targets $C_i \cdot p/1000$ wall-clock time per job (with p drawn from a per-task table). Because the loop is time-bounded rather than work-bounded, C_i is the maximum CPU time an alone-running job consumes; the experiments therefore quantify scheduling-induced timing behavior under a controlled execution profile, not WCET characteristics of an application.

We use FreeRTOS for the experiments (specifically `SW32G_FreeRTOS_11.1.0_4.0.3_CD3_D2410`) configured to run in AMP mode, with execution restricted to a single Cortex-M7 core. For debugging and tracing, the Lauterbach Trace32 environment was employed together with a PowerDebug X50 hardware probe. Program tracing data was collected through ARM's on-chip CoreSight infrastructure, specifically the ETM and its associated Embedded Trace Buffers (ETB). For the experiments, FreeRTOS was configured and deployed on the NXP S32G-VNP-RDB2 board. On the Cortex-M7 cores, the system tick is handled by the hardware SysTick timer, which is part of the Cortex-M architecture.

We measure the overall overhead introduced by context switches during a defined period of time. The results captured both the number of context switches and their cumulative duration over an entire period. Although the context switch latency is expected to be higher in the tickless design (c.f. Section IV), these activations occur on an on-demand basis, thereby reducing the number of required context switches. In the tick based case, when tasks of equal priority reach the end of the current timeslice, the system tick triggers a context switch. The mechanism operates as follows:

- The RTOS configures the SysTick timer (a dedicated hardware timer built into the Cortex-M architecture) to expire at the defined tick rate.
- When the timer expires, it generates an interrupt request.
- The Nested Vectored Interrupt Controller (NVIC) receives the request and forwards the interrupt to the processor.
- The processor branches execution to the exception vector table to invoke the corresponding ISR.
- The SysTick ISR performs tick accounting and, if a reschedule is required, it sets the `PendSV` exception as pending.
- The RTOS searches across the priority-sorted task queues to identify the next task in the `READY` state from the highest-priority queue.
- The RTOS updates the scheduling state, marking the current task as `READY` and the selected task as `RUNNING`.
- After SysTick ISR completes, the Cortex-M exception logic triggers the `PendSV` handler (the lowest-prio exception), ensuring the context switch does not preempt other interrupts.
- The `PendSV` handler saves the current task context, restores the context of the selected task, and finalizes the switch.

For the proposed tickless design, the overall mechanism is nearly identical, except that the system tick does not trigger the context switch. Instead, the TTS timer initiates the context switch precisely at the slot boundaries, thereby reducing and

optimizing the number of required context switches.

A. Default tick–based

For the purpose of this analysis, we assumed that tasks do not yield voluntarily, in order to avoid relying on a fully cooperative execution model. In the tick-based configuration, we set the FreeRTOS tick period of $50\mu s$. FreeRTOS uses a fixed-priority preemptive scheduling policy with Round-Robin time slicing among tasks of equal priority.

TABLE I
ABSOLUTE ACCUMULATED OVERHEAD IN FREERTOS.

Exp. duration	Tick Rate	Overhead (-O0)	Overhead (-Os)
$1\,s$	$50\,\mu s$	$25\,ms$	$19.6\,ms$
$10\,s$	$50\,\mu s$	$250\,ms$	$196\,ms$
$100\,s$	$50\,\mu s$	$2.5\,s$	$1.96\,s$

As shown in Table I, this approach is impractical and does not scale well, particularly when the task set includes at least one task with a very short WCET imposing low tick values.

B. Fully cooperative mode

In this experiment, FreeRTOS is configured to not preempt and rely on tasks voluntarily yielding when they finish and before the next task is required to run.

TABLE II
FREERTOS PER-CONTEXT-SWITCH OVERHEAD INCURRED WHEN TASKS VOLUNTARILY YIELD.

Overhead (-O0)	Overhead (-Os)
$0.622\,\mu s$	$0.500\,\mu s$

In this configuration, the tasks are mostly independent and are not affected by the configured system tick value (cf. Table II). The yield-related context–switch overhead values can be considered lower bounds, as they correspond to a reduced form of the tick-triggered context switches and do not include the interrupt and its associated ISR.

C. Proposed tick–less

In the tick–less case, the tick period was kept at the default value of 1 ms. This value could also be increased without affecting the execution of the experiment tasks. However, in order to preserve the default configuration of the RTOS–provided software timers and other native timing–related features, the default tick period was retained. The tasks were configured to run in a pre–defined sequence and triggered at precise moments, respecting the data dependencies between them and guaranteeing the Freedom-from-interference (FFI) property.

Similarly to the cooperative mode, the overhead is independent of the configured system tick and depends only on the number of context switches. Although the per-context-switch overhead is higher, $6.5\mu s$ in our measurements, task dispatch is enforced at the schedule-table boundaries, so an overrun in one slot does not shift later slot releases. We note that deadline misses still depend on the configured execution budgets and slot lengths and not purely on the overrun protection.

We measured the FreeRTOS per-context-switch overhead with TTS at $6.5\mu s$. The default tick-driven configuration scales poorly when tight execution times impose high tick rates (cf. Table I), leading to substantial accumulated overhead and potential deadline violations. Cooperative execution lowers the per-switch overhead (c.f. Table II) but cannot guarantee timing without assuming task cooperation. In contrast, the proposed TTS approach enforces task releases at precise instants, providing deterministic dispatch even though the per-switch cost is higher ($6.5\mu s$). Crucially, its overhead is independent of the configured tick value and depends only on the number of required task context switches.

Overall, the results expose a design-time trade-off between tick frequency, context-switch cost, and temporal guarantees:

- **Tick-based (FP + RR)** offers compatibility and simplicity but scales poorly as tick periods approach the minimum task WCET, inflating overhead and jeopardizing deadlines.
- **Cooperative** minimizes switch cost but relies on voluntary yields, weakening FFI guarantees and robustness under load.
- **Proposed TTS** provides deterministic releases and schedule-table isolation with overhead that is stable with respect to the system tick, making it preferable when accuracy and predictability dominate cost.

D. Time-Triggered vs. Fixed Priority Scheduling

We now compare TT vs. FP in terms of execution jitter. For this, we instantiate 15 periodic tasks with periods $10, 10, 20, 20, 25, 40, 50, 50, 80, 100, 100, 125, 200, 250, 500$ ms, giving a hyperperiod of $H = 2000$ ms. The runtime payload is a *time-bounded busy-wait loop* polled against the SoC hardware timer (STM): the loop reads the counter, performs filler floating-point work, and exits when the counter reaches a target end value $\text{budget}_{i,k} = C_i \cdot p/1000$, where p is drawn from a per-task deterministic sequence over a 100-entry table with Gumbel distribution similar to [13]. Two load profiles are evaluated: **Baseline Load** (Mode 0), with $p \leq 1000$ for every table entry (i.e., every job stays within C_i); and **Transient Peak Load** (Mode 1), with 2 heavy entries ($p \in \{3000, 5000\}$) and the remaining 98 entries equal to Mode 0. Mode 1 table thus selects $\sim$2% of jobs that *request* 3×–5× their nominal budget. Both runs use identical seeds and a common table, so the per-job permille sequence is reproducible across modes and across schedulers.

a) FP start jitter behavior: For FP scheduling we use Rate-Monotonic priority assignment: task τ_i receives priority $\texttt{configMAX_PRIORITIES} - 2 - i$, where tasks are indexed in non-decreasing order of period ($T_0 \leq T_1 \leq \cdots \leq T_{14}$). Equal-period ties are broken by index, so the lower-indexed task has the higher priority. For TT scheduling we run the same task set on the proposed TT layer with a statically generated schedule table consisting of 914 timeslices uniformly spaced at 1 ms over the 2 s hyperperiod (one resync slot at offset 0 and 913 task slots at offsets 1 ms, 2 ms, ...). The schedule table is produced offline by an automatic generator that places the per-task instances at the chosen 1 ms grid.

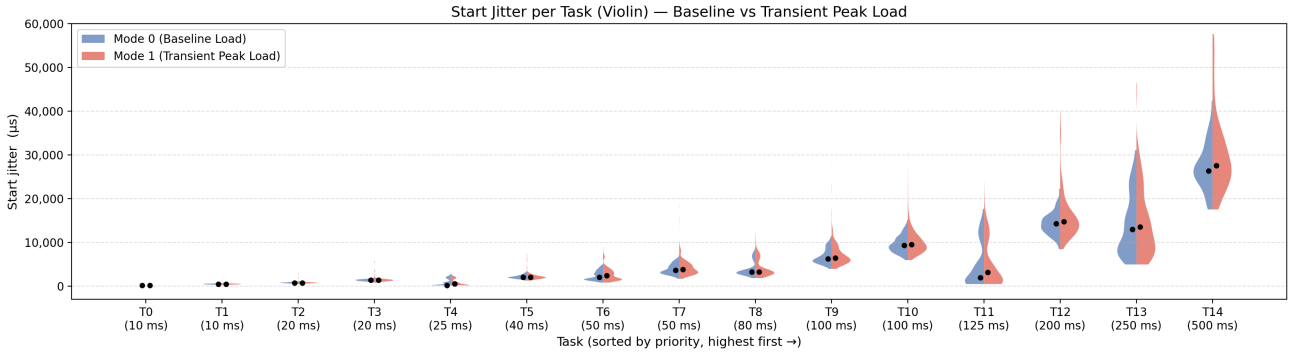


Fig. 5. FP scheduling: start jitter per task. Left half (blue): nominal mode; right half (red): overload mode (2% of jobs at 3×–5× WCET).

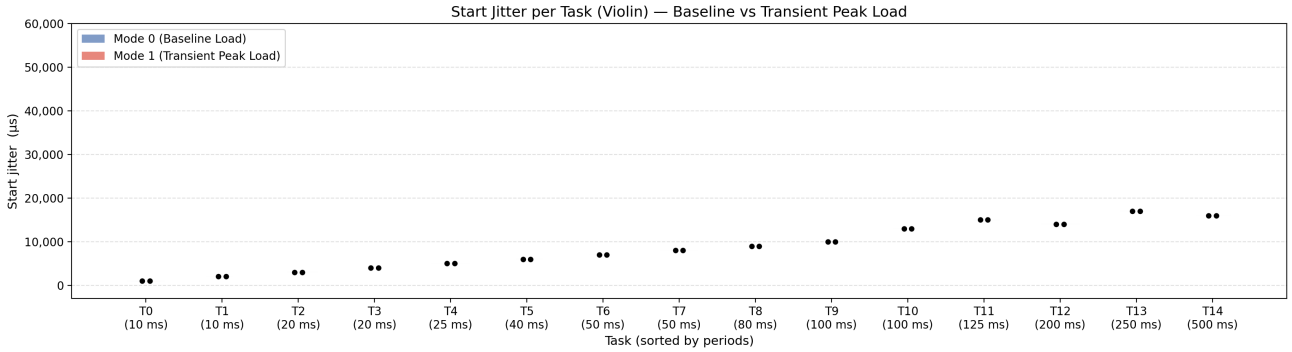


Fig. 6. TT scheduling: start jitter per task using the same task set and modes as Figure 5. Each task's distribution is concentrated within about 3.2 μs of its scheduled slot offset. The two halves overlap completely, indicating insensitivity to the Mode 1 transient peak load.

For each released job we record the ideal release $r_{i,k} = \phi_0 + k \cdot T_i$ relative to a phase reference ϕ_0 (captured at scheduler start; per-task in the FP path, common in the TT path) and the actual start time $s_{i,k}$ (the moment the task begins executing after being released). The start jitter is $J_{i,k} = s_{i,k} - r_{i,k}$. Under FP, $J_{i,k}$ captures the cumulative interference from higher-priority releases between the period boundary and the start of τ_i. Under TT, $J_{i,k}$ captures both the deterministic offset of τ_i's slot inside the hyperperiod and any timer-IRQ / context-switch latency. The run contains 15,000 job records per scheduler/mode, corresponding to approximately 32.9 s of releases for this task set. The highest-frequency tasks contribute approximately 3285–3288 samples each, while the lowest-frequency task contributes 65–66 samples.

Figure 5 presents the start-jitter distributions for all 15 tasks under both nominal (Mode 0) and overload (Mode 1) conditions under FP scheduling. For each task τ_i, the left half (blue) shows the density of start-jitter observations in nominal mode, while the right half (red) shows the transient peak mode. Tasks are ordered left-to-right by decreasing priority. The upper envelope of the jitter distribution generally grows with task index (and therefore with period and decreasing priority), consistent with Rate-Monotonic analysis [25], i.e., a job's actual start can only be delayed by higher-priority releases, and lower-priority tasks accumulate interference from a larger set of preempting tasks. The medians are not expected to be strictly monotone, because a task's observed start jitter also depends on its release phase relative to the higher-priority release pattern. In particular, τ_{11} (125 ms) and τ_{13} (250 ms) often release at offsets that avoid the densest high-priority clusters, whereas neighboring periods such as 200 ms align more frequently with releases from the 10–100 ms tasks. Their maxima still increase when those offsets coincide with dense clusters, but their typical values can lie below adjacent tasks. The highest-priority task τ_0 exhibits the tightest distribution (187–191 μs, std-dev $\approx$ 0.9 μs) and is essentially limited to context-switch and scheduler overhead, since no other task can preempt it. The next task τ_1 shares the same 10 ms period but has lower priority and waits behind τ_0 and hence its jitter spreads to 312–666 μs in Mode 0 and to 312–2534 μs in Mode 1. The lowest-priority task τ_{14} ranges from about 17.5 ms in its most favorable observed releases to $\sim$57.6 ms under Mode 1. Only τ_0 retains an identical Mode 0/Mode 1 distribution, since no overload job can preempt it from above. Already from τ_1 onward the overload half of the violin extends noticeably further than the nominal half and we observe that the per-task max jitter grows from 666 $\rightarrow$ 2534 μs for τ_1, 1109 $\rightarrow$ 2974 μs for τ_2, 1985 $\rightarrow$ 5717 μs for τ_3, and 2750 $\rightarrow$ 6455 μs for τ_4.

b) TT start jitter behavior: Figure 6 shows the same experiment with our TT scheduling layer. Each task's jitter is dominated by the deterministic offset of its slot in the

hyperperiod: in the TT schedule, τ_i's first slot of every period sits at the 1 ms grid, so the median jitter is approximately constant per task and equal to that offset (e.g., $\tau_1 \approx 2008\,\mu$s, $\tau_2 \approx 3008\,\mu$s, $\tau_{14} \approx 16008\,\mu$s). Around that offset the per-task spread is essentially flat and the standard deviation is below $0.32\,\mu$s for all tasks and the peak-to-peak range across the run is at most about $3.2\,\mu$s. The residual variance reflects timer-IRQ latency and context-switch cost rather than interference from other tasks. In Figure 6 we see that the $\sim 2\%$ of jobs that request $3\times$–$5\times$ their nominal budget have no measurable effect on the start jitter of any task. This is a direct consequence of the slot discipline where at each slot boundary the TT layer demotes the previous task to background priority and promotes the next one, so an oversized-budget job of τ_i cannot delay the dispatch of τ_{i+1}. The same Mode 1 table that produced multi-millisecond tails for the lower-priority FP tasks (Figure 5) leaves the TT start-time distributions unchanged.

c) FP vs. TT: Both runs use the same hardware, the same task set and the same per-job permille sequence and only the scheduler differs. Three main differences stand out:

- *Magnitude.* The lowest-priority FP task (τ_{14}) sees jitter up to 57.6 ms under overload, while in TT the largest observed jitter is bounded by the deterministic slot offset plus dispatch latency (e.g., τ_{13} reaches about $17{,}010.5\,\mu$s).
- *Variance.* FP per-task standard deviations generally increase with decreasing priority, although not monotonically for every adjacent pair, and reach $\sim$ 8.7 ms for τ_{14} under Mode 1, whereas every TT task except τ_0 has a standard deviation below $0.32\,\mu$s in both modes.
- *Load sensitivity.* FP per-task max jitter increases with overload from τ_1 onward (e.g., τ_3: 2.0 $\rightarrow$ 5.7 ms, τ_{11}: 17.7 $\rightarrow$ 36.3 ms). TT per-task max jitter remains the same in Mode 0 and Mode 1.

The experiment shows a principal tradeoff between TT and FP. TT trades the flexibility of FP for design-time predictability in the sense that the start time of the task becomes a property of the schedule table rather than of the runtime workload. The experiments also show that our TT scheduler implementation behaves as expected in a real-time environment.

VI. Conclusion

We presented a portable time-triggered scheduling (TTS) layer that retrofits HW-timer–driven execution into unmodified tick-based RTOSes without requiring kernel changes. By combining static, time-triggered activation with standard fixed-priority mechanisms, the approach provides deterministic, sub-millisecond task release jitter relative to the scheduled slot while preserving compatibility with legacy workloads and default RTOS timing services. Our evaluation shows that conventional tick-based scheduling scales poorly at high tick rates and cooperative execution lowers overhead but weakens FFI guarantees, whereas the proposed TTS ensures deterministic releases with tick-independent overhead, clearly exposing the trade-off between context-switch cost, tick frequency, and dispatch-time predictability.

Copyright & Trademark

NXP and the NXP logo are trademarks of NXP B.V. All other product or service names are the property of their respective owners. Arm, Cortex, Cortex-R4F are trademarks and/or registered trademarks of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere. The related technology may be protected by any or all of patents, copyrights, designs and trade secrets. All rights reserved.

References

[1] AWS open source, "Freertos," Online, accessed: 2026-01-21. [Online]. Available: https://www.freertos.org/

[2] Linux Foundation, "Zephyr," Online, accessed: 2026-01-21. [Online]. Available: https://zephyrproject.org/

[3] PX5 RTOS Inc., "Px5," Online, accessed: 2026-01-21. [Online]. Available: https://px5rtos.com/

[4] J. Beningo, "Real-time operating system (rtos) 2024 performance report," Beningo Embedded Group, LLC, Tech. Report, 2024.

[5] H. Kopetz, "Time-triggered real-time computing," *Annual Reviews in Control*, vol. 27, no. 1, pp. 3–13, 2003.

[6] J. Xu, "Satisfying complex timing constraints with pre-run-time scheduling," *IFAC Proceedings Volumes*, vol. 35, no. 1, pp. 273–278, 2002.

[7] H. Lonn and J. Axelsson, "A comparison of fixed-priority and static cyclic scheduling for distributed automotive control applications," in *Proc. ECRTS*, 1999.

[8] F. Sagstetter, S. Andalam, P. Waszecki, M. Lukasiewycz, H. Stähle, S. Chakraborty, and A. Knoll, "Schedule integration framework for time-triggered automotive architectures," in *Proc. DAC*. ACM, 2014.

[9] A. Mehmed, W. Steiner, and M. Rosenblattl, "A time-triggered middleware for safety-critical automotive applications," in *Ada-Europe*, 2017.

[10] R. Ernst, S. Kuntz, S. Quinton, and M. Simons, "The Logical Execution Time Paradigm: New Perspectives for Multicore Systems," *Dagstuhl Reports*, vol. 8, no. 2, 2018.

[11] B. Akesson, M. Nasri, G. Nelissen, S. Altmeyer, and R. I. Davis, "A comprehensive survey of industry practice in real-time systems," *Real-Time Systems*, vol. 58, no. 3, pp. 358–398, Sep 2022.

[12] S. Craciunas, R. Serna Oliver, and V. Ecker, "Optimal static scheduling of real-time tasks on distributed time-triggered networked systems," in *Proc. ETFA*, 2014.

[13] P. Karachatzis, J. Ruh, and S. S. Craciunas, "An evaluation of time-triggered scheduling in the linux kernel," in *Proc. RTNS*. ACM, 2023.

[14] TTTech Computertechnik AG, "Automated Driving Offering," https://www.tttech-auto.com/motionwise, 2018, accessed: 06.02.2026.

[15] M. Sokcevic, "Partitioned Complexity," https://www.tttech-auto.com/knowledge-platform/partitioned-complexity, 2020, 06.02.2026.

[16] J. M. Calandrino, H. Leontyev, A. Block, U. C. Devi, and J. H. Anderson, "$LITMUS^{RT}$: A testbed for empirically comparing real-time multiprocessor schedulers," in *Proc. RTSS*, 2006.

[17] FreeRTOS, "Low power tickless idle mode," https://www.freertos.org/low-power-tickless-rtos.html, 2023, accessed: 2026-04-28.

[18] R. Segger, "Real-time operating system with a cpu cycle time base," Patent Application US20 230 130 826A1, 2023.

[19] J. Beningo. A new rtos approach to achieve sub-millisecond timing resolution. Beningo Embedded Group. Accessed: 2026-01-21. [Online]. Available: https://www.beningo.com/a-new-rtos-approach-to-achieve-sub-millisecond-timing-resolution/

[20] A. A. Fröhlich, G. Gracioli, and J. F. Santos, "Periodic timers revisited: The real-time embedded system perspective," *Computers & Electrical Engineering*, vol. 37, no. 3, pp. 365–375, 2011.

[21] K. Heider, C. Hakert, K.-H. Chen, and J.-J. Chen, "Lazytick: Lazy and efficient management of job release in real-time operating systems," *ACM Trans. Embed. Comput. Syst.*, vol. 24, no. 5s, 2025.

[22] B. B. Brandenburg and J. H. Anderson, "On the implementation of global real-time schedulers," in *Proc. RTSS*. IEEE, 2009.

[23] J. Xu and D. Parnas, "Priority scheduling versus pre-run-time scheduling," *IFAC Proceedings Volumes*, vol. 31, no. 14, pp. 53–60, 1998.

[24] M. Lukasiewycz, R. Schneider, D. Goswami, and S. Chakraborty, "Modular scheduling of distributed heterogeneous time-triggered automotive systems," in *Proc ASPDAC*, 2012.

[25] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, p. 46–61, Jan. 1973.